

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
data = pd.read_csv('/content/Social_Network_Ads.csv')
```

```
print("Dataset loaded!")
print("Shape:", data.shape)
data.head()
```

Dataset loaded!
Shape: (400, 5)

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

```
print(data.columns)
print("\nData types:\n", data.dtypes)
```

```
Index(['User ID', 'Gender', 'Age', 'EstimatedSalary', 'Purchased'], dtype='object')
```

```
Data types:
User ID      int64
Gender       object
Age          int64
EstimatedSalary  int64
Purchased    int64
dtype: object
```

```
# Gender column is categorical – convert to numerical
data['Gender'] = data['Gender'].map({'Male': 1, 'Female': 0})
```

```
data.head()
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	1	19	19000	0
1	15810944	1	35	20000	0
2	15668575	0	26	43000	0
3	15603246	0	27	57000	0
4	15804002	1	19	76000	0

```
data.isnull().sum()
```

```
0
User ID    0
Gender     0
Age        0
EstimatedSalary  0
Purchased  0
```

```
dtype: int64
```

```
# Independent variables (features)
x = data[['Age', 'EstimatedSalary']].values

# Dependent variable (target)
```

```
y = data['Purchased'].values
```

```
print("X shape:", x.shape)  
print("y shape:", y.shape)
```

```
X shape: (400, 2)  
y shape: (400,)
```

```
from sklearn.model_selection import train_test_split
```

```
xtrain, xtest, ytrain, ytest = train_test_split(  
    x, y,  
    test_size=0.2,  
    random_state=0)
```

```
print("Train size:", xtrain.shape)  
print("Test size :", xtest.shape)
```

```
Train size: (320, 2)  
Test size : (80, 2)
```

```
from sklearn.preprocessing import StandardScaler
```

```
sc = StandardScaler()  
xtrain = sc.fit_transform(xtrain)  
xtest = sc.transform(xtest)
```

```
print("Feature scaling done!")
```

```
Feature scaling done!
```

```
from sklearn.linear_model import LogisticRegression
```

```
logreg = LogisticRegression()  
  
logreg.fit(xtrain, ytrain)  
  
print("Model trained successfully!")
```

```
Model trained successfully!
```

```
# Predict for test data  
y_pred = logreg.predict(xtest)
```

```
print("Predicted :", y_pred[:10])  
print("Actual    :", ytest[:10])
```

```
Predicted : [0 0 0 0 0 0 0 1 0 0]  
Actual    : [0 0 0 0 0 0 0 1 0 0]
```

```
from sklearn.metrics import confusion_matrix
```

```
cm = confusion_matrix(ytest, y_pred)
```

```
print("Confusion Matrix:")  
print(cm)
```

```
Confusion Matrix:  
[[57  1]  
 [ 5 17]]
```

```
from sklearn.metrics import (confusion_matrix, accuracy_score,  
                             precision_score, recall_score)
```

```
accuracy = accuracy_score(ytest, y_pred)  
error_rate = 1 - accuracy  
precision = precision_score(ytest, y_pred)  
recall = recall_score(ytest, y_pred)
```

```
from sklearn.metrics import (confusion_matrix, accuracy_score,  
                             precision_score, recall_score)
```

```
accuracy = accuracy_score(ytest, y_pred)
```

```
error_rate = 1 - accuracy
precision = precision_score(ytest, y_pred)
recall = recall_score(ytest, y_pred)

print(f"\nAccuracy : {accuracy:.4f}")
print(f"Error Rate : {error_rate:.4f}")
print(f"Precision : {precision:.4f}")
print(f"Recall : {recall:.4f}")
```

```
Accuracy : 0.9250
Error Rate : 0.0750
Precision : 0.9444
Recall : 0.7727
```

```
print("Confusion Matrix:")
print(cm)
print()
print(f"TP = {TP}")
print(f"TN = {TN}")
print(f"FP = {FP}")
print(f"FN = {FN}")
```

```
Confusion Matrix:
[[57  1]
 [ 5 17]]
```

```
TP = 17
TN = 57
FP = 1
FN = 5
```